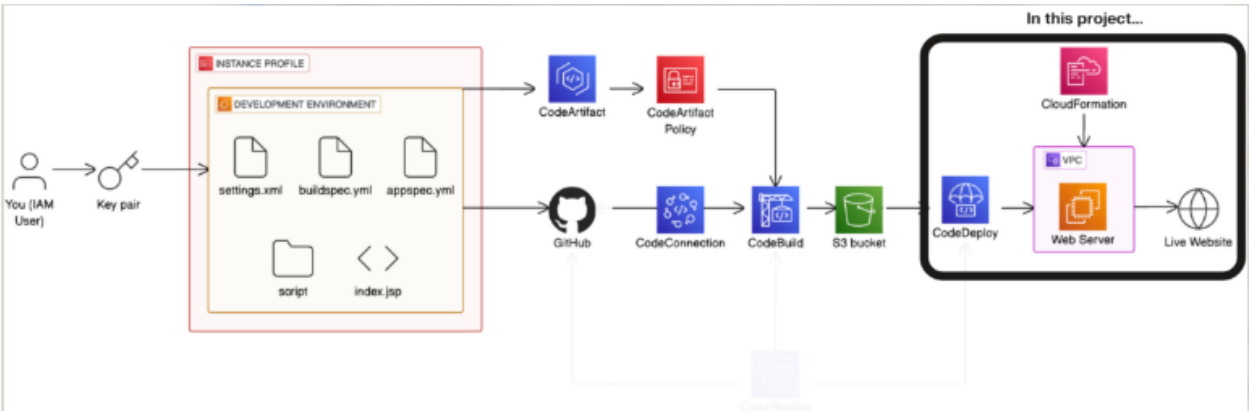
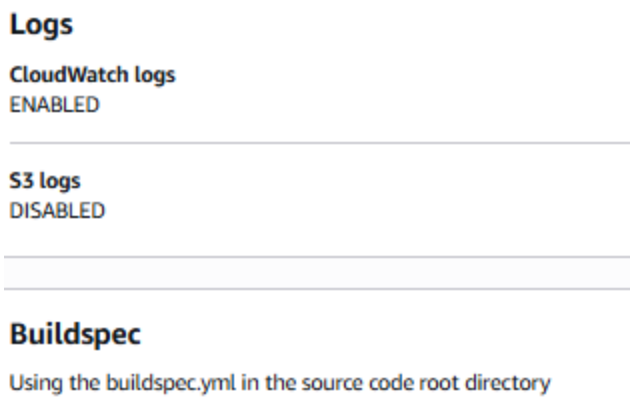
# AWS CI/CD Pipeline Pt3

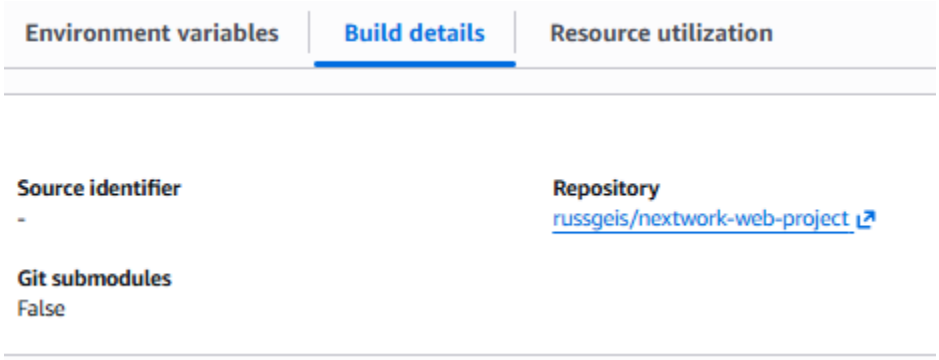



Let's get ready to run our first build! It's okay if it fails initially – that's part of the learning process.

- Navigate to your newly created CodeBuild project nextwork-devops-cicd.
- Click the **Start build** button.
- You will be redirected to the build execution page.
- You should see the build status change to **In progress**
- Wait for the build to complete.
- Oooo, looks like the build failed!
- By checking the logs and phase details, we can pinpoint exactly where and why our build is failing.
- Select the **Phase details** tab to understand the failure.
- Aha, the DOWNLOAD_SOURCE phase has failed with the error message YAML_FILE_ERROR: YAML file does not exist
- Click on the **Build details** tab.
- Scroll down to the **Buildspec** section.
- Let's make sure that it says **Using the buildspec.yml in the source code root directory**.




- This confirms that CodeBuild *is* looking for **buildspec.yml** in the root directory of our web app's nextwork-web-project code repository... we haven't created that file yet!
- In fact, let's check our GitHub repository. Select the **Repository** link in your CodeBuild project




- there's no buildspec.yml file - it should be in the root of your web app repository 🙈 Let's create it now
- Open your project in VS Code or your preferred code editor.
- Select **File** > **Open Folder...**
- Open your nextwork-web-project folder.
- In VS Code, in the Explorer pane, create a new file on the project root (nextwork-web-project
- Enter **buildspec.yml** as the file name
- Paste the following code for buildspec.yml:

```yaml
version: 0.2

phases:
  install:
    runtime-versions:
      java: corretto8
  pre_build:
    commands:
      - echo Logging in to AWS CodeArtifact...
      - CODEARTIFACT_AUTH_TOKEN=`aws codeartifact get-authorization-token --
      - export CODEARTIFACT_AUTH_TOKEN
  build:
    commands:
      - echo Build started on `date`
      - mvn clean install -s settings.xml
  post_build:
    commands:
      - echo Build completed on `date`
      - echo Packaging artifacts...
      - mvn package -s settings.xml
artifacts:
  files:
    - target/*.war
  discard-paths: no
```

In the buildspec.yml file:

- Replace the placeholder AWS Account ID 123456789012 with your actual AWS Account ID.
- Check that the region code is correct! Update the region section from --region us-east-2 to the AWS region you're using
- Save the buildspec.yml file

Confirm that the small circle indicator on the buildspec.yml file tab disappears after saving, which means the file is saved.

- Now, we need to commit and push the buildspec.yml file to your GitHub repository so CodeBuild can access it.
- Open the terminal in VS Code (Ctrl+or Cmd+).
- Run the following Git commands:
    - git add .
    - git commit -m "Adding buildspec.yml file"
    - git push

```
[ec2-user@ip-172-31-18-246 nextwork-web-project]$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1019 bytes | 1019.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/russgeis/nextwork-web-project.git
   f3418c8..19f4220  master -> master
[ec2-user@ip-172-31-18-246 nextwork-web-project]$
```

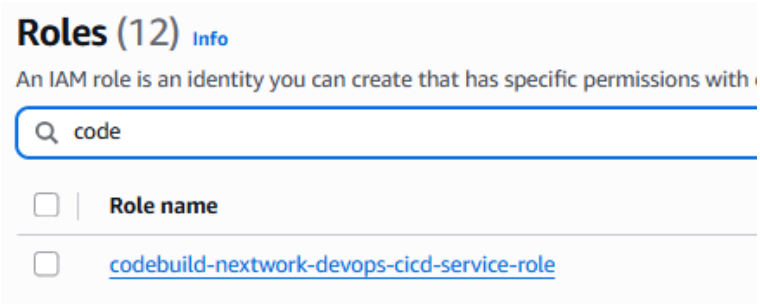
- Go to your GitHub repository nextwork-web-project in your browser and refresh the page to verify that buildspec.yml is now present in the repository.

**Retry your build**

- Return to the CodeBuild console in your browser.
- Retry the build by clicking the **Retry build** button.
- The build status should now show **In progress**. Wait for this build to complete
- We failed again!
- Check the **Phase details** tab again after this retry.
- You will likely see that the DOWNLOAD_SOURCE phase succeeded this time, but other phases might have failed!

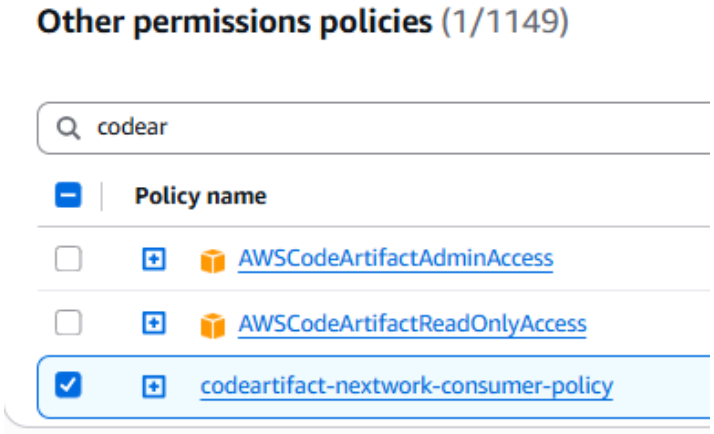
To fix this, we need to grant CodeBuild's IAM role the permission to access CodeArtifact.

- Head to the IAM console.
- In the IAM console, select **Roles** in the left navigation menu.
- In the roles search bar, type codebuild to filter the roles.




- Select the role that starts with codebuild-nextwork-devops-cicd-service-role. This is the new service role that CodeBuild created when we set up our build project.
- Select your CodeBuild service role.
- Click on the **Add permissions** button

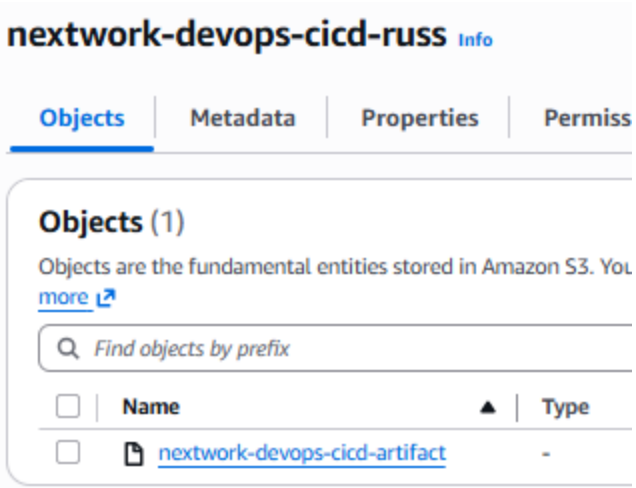
- Choose **Attach policies**.
- In the **Filter policies** search bar, type codeartifact-nextwork-consumer-policy.
- Check the checkbox next to the policy named codeartifact-nextwork-consumer-policy




- 
- After selecting the policy, click the **Add permissions** button.
- You should see a green banner confirming Policy was successfully attached to role. and the policy should now be listed under the **Permissions policies** of your CodeBuild service role.
- Return to the CodeBuild console, navigate to your nextwork-devops-cicd project.
- Retry the build one more time by clicking **Retry build**
- NICE - the build status should now be **Succeeded** with a green checkmark, indicating a successful CI pipeline run!

Let's check if our build process has successfully created and stored the artifact 🏃

- To verify the successful build, let's check if the build artifact was correctly uploaded to our S3 bucket.
- Head to the S3 console.
- Select **Buckets** from the left navigation menu.
- Select the nextwork-devops-cicd bucket you created earlier.
- You should now see the artifact nextwork-devops-cicd-artifact.zip listed in your bucket




## Launch EC2 with CloudFormation

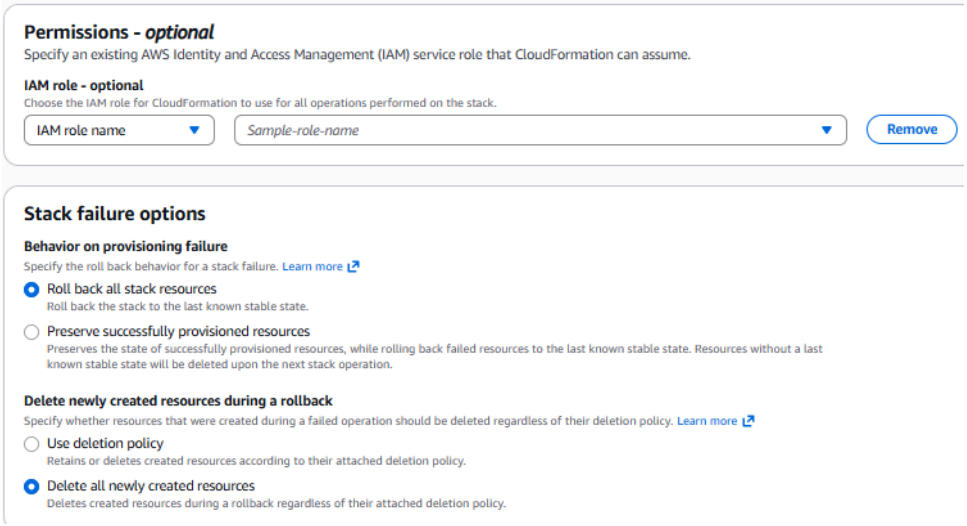
- Head to the CloudFormation console.
- Select **Create stack**.
- Select **With new resources (standard)** from the dropdown menu.
- Select **Template is ready**.
- Select **Upload a template file**
- Select **Choose file**
- Upload the following CloudFormation template: nextworkwebapp.yaml
- Verify the file nextworkwebapp.yaml is uploaded and select **Next**

### Configure Stack Details

- Select **Next**.
- Enter NextWorkCodeDeployEC2Stack as the **Stack name**.
- Next, we'll need to add our IP address to the template.
- Head to https://checkip.amazonaws.com/ and copy your IP address.
- Paste your IP address into the **MyIP** parameter field and add /32 at the end
- Select **Next**.

### Configure Stack Options

- In **Configure stack options**, under **Stack failure options**, select **Roll back all stack resources**.
- Next, select **Delete all newly created resources**.




- Scroll down to **Capabilities** and check the box **I acknowledge that AWS CloudFormation might create IAM resources**.
- **Review and Submit**
- Let's review our stack's details!
- Template: **Template is ready**
- Stack name: NextWorkCodeDeployEC2Stack
- Parameters: MyIP is filled with your IP and /32
- Rollback on failure: **Activated**
- Delete all newly created resources during rollback: **Activated**

### Monitor Stack Creation

- CloudFormation is now launching the stack in the background! This process will create the EC2 instance and networking resources in the background.
- While we're waiting
- Let's check out the **Resources** tab.
- You can see a list of the resources being created!
- Next, you can also check the **Events** tab.
- You can watch your CloudFormation stack's events in real-time in the **Events** tab - just keep refreshing it!
- Wait for the stack's status to become CREATE_COMPLETE

### Prepare Deployment Scripts

### Create Scripts Folder

- In your VS Code, create a new folder at the root of your project directory

- Name the folder **scripts**
- Inside the scripts folder, create a new file.
- Let's name the file **install_dependencies.sh**
- Add the following content to install_dependencies.sh

```bash
#!/bin/bash
sudo yum install tomcat -y
sudo yum -y install httpd
sudo cat << EOF > /etc/httpd/conf.d/tomcat_manager.conf
<VirtualHost *:80>
  ServerAdmin root@localhost
  ServerName app.nextwork.com
  DefaultType text/html
  ProxyRequests off
  ProxyPreserveHost On
  ProxyPass / http://localhost:8080/nextwork-web-project/
  ProxyPassReverse / http://localhost:8080/nextwork-web-project/
</VirtualHost>
EOF
```

Save the install_dependencies.sh file by pressing **Ctrl+S**. You should see the round circle disappear.

### Create start_server.sh

- Still inside the scripts folder, let's create a new file again
- This time, let's name the file start_server.sh
- Add the following content to start_server.sh

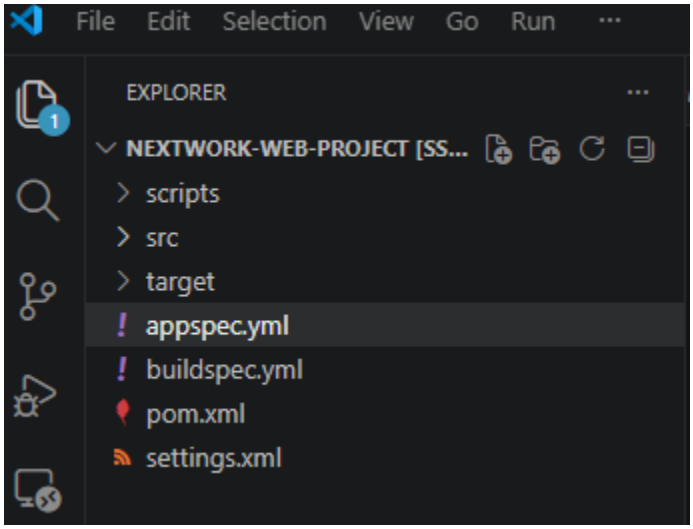
```bash
#!/bin/bash
sudo systemctl start tomcat.service
sudo systemctl enable tomcat.service
sudo systemctl start httpd.service
sudo systemctl enable httpd.service
```

### Create stop_server.sh script

- Inside the scripts folder, create a new file named stop_server.sh
- Add the following content to stop_server.sh

### Create appspec.yml

- Create a new file, but this time at the **root** of your project.
- Make sure this file is NOT inside the scripts folder!
- Let's name the file appspec.yml
- Add the following content to appspec.yml




### Update buildspec.yml File

- Open buildspec.yml and modify the artifacts section to include appspec.yml and the scripts folder

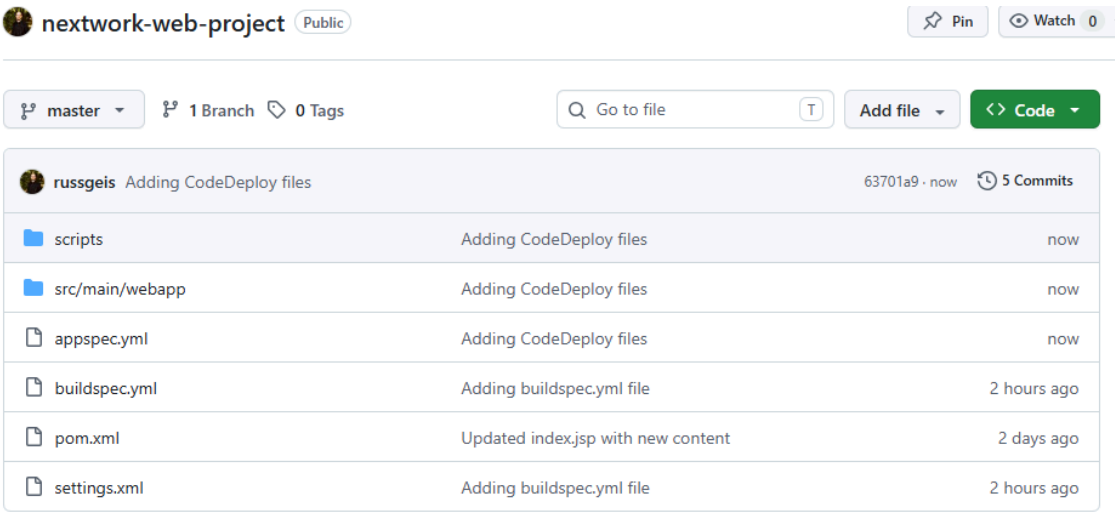
```yaml
artifacts:
  files:
    - target/nextwork-web-project.war
    - appspec.yml
    - scripts/**/*
  discard-paths: no
```

-

### Commit and Push Changes to GitHub

- Open the terminal in VS Code.
- Stage all changes using **git add .**
- Commit changes with the message "Adding CodeDeploy files" using **git commit -m "Adding CodeDeploy files"**
- Push changes to your GitHub repository using **git push**

- Head to your GitHub repository in the browser. Let's check that the scripts folder and appspec.yml file are in your repository!




### Create CodeDeploy Application

- Head to the CodeDeploy console.
- Select **Applications** in the left hand navigation menu
- Select **Create application**
- Enter nextwork-devops-cicd as the **Application name**
- Choose **EC2/On-premises** as the **Compute platform**.
- That's it! Select **Create application**

### Create Deployment Group

- Select **Create deployment group**
- Enter **nextwork-devops-cicd-deploymentgroup** as the **Deployment group name**.

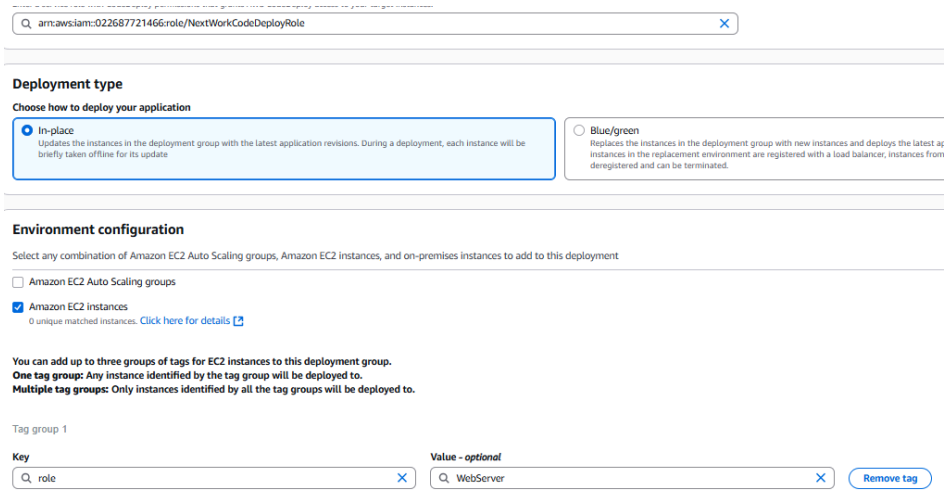
### Create an IAM Role for CodeDeploy

- Head to the IAM console
- In the IAM console, select **Roles** from the left hand navigation bar
- Select **Create role**.
- Choose **CodeDeploy** as the service and select **CodeDeploy** as the use case.
- Select **Next**.

- You'll notice the AWSCodeDeployRole default policy is suggested already - nice! That's all we need.
- Click on the plus button to take a look at the permissions this grants. There are many actions that we're allowing in this policy!
- Phew, this saves us the time from defining all these permission policies ourselves.
- Select **Next**.
- Expand the AWSCodeDeployRole policy to review its permissions.
- Review the EC2 permissions within the policy
- Select **Next**.
- Enter NextWorkCodeDeployRole as the **Role name**.
- Add a description to help you remember why you created this role.
    - Allows CodeDeploy to call AWS services such as Auto Scaling on your behalf.
    - Created as a part of NextWork's CI/CD Pipeline series.
- Review the **Permissions policies** and make sure AWSCodeDeployRole is attached.
- Select **Create role**.

**Select Service Role in CodeDeploy**

- Head back to the **CodeDeploy deployment group** configuration tab.
- Looks like we'll need to refresh the page to use the new service role!
- Refresh the page, and re-enter **nextwork-devops-cicd-deploymentgroup** as **Deployment group name**.
- Select the newly created NextWorkCodeDeployRole as the **Service role**.
- Choose **In-place** as the **Deployment type**
- Under **Environment configuration**, select **Amazon EC2 instances**.
- In **Tag group 1**, enter **role** as the Key.
- Enter **webserver** as the Value
- Check the line below your tag settings - you might notice that **1 unique matched instance** is found.



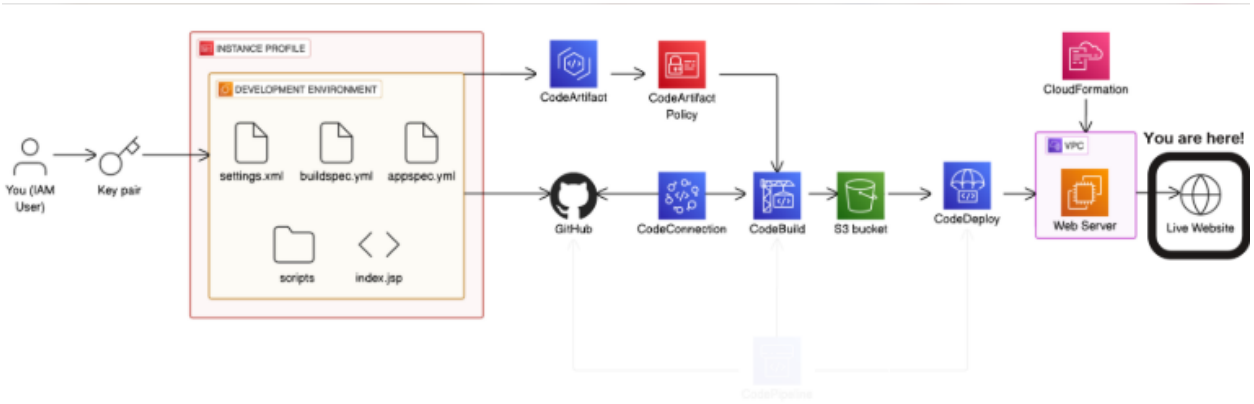

- Click **Click here for details** to view the matched instance.
- You'll see an EC2 instance called **NextWorkCodeDeployEC2Stack::WebServer** - that's the EC2 instance we launched from our CloudFormation template. This confirms to us that the web app will be deployed onto that instance.

### Configure Agent and Deployment Settings

- Now let's head back to your CodeDeploy **Deployment group** set up.
- Under **Agent configuration with AWS Systems Manager**, select **Now and schedule updates** and **Basic scheduler** with **14 Days**.
- Deselect **Enable load balancing**.
- Select **Create deployment group**

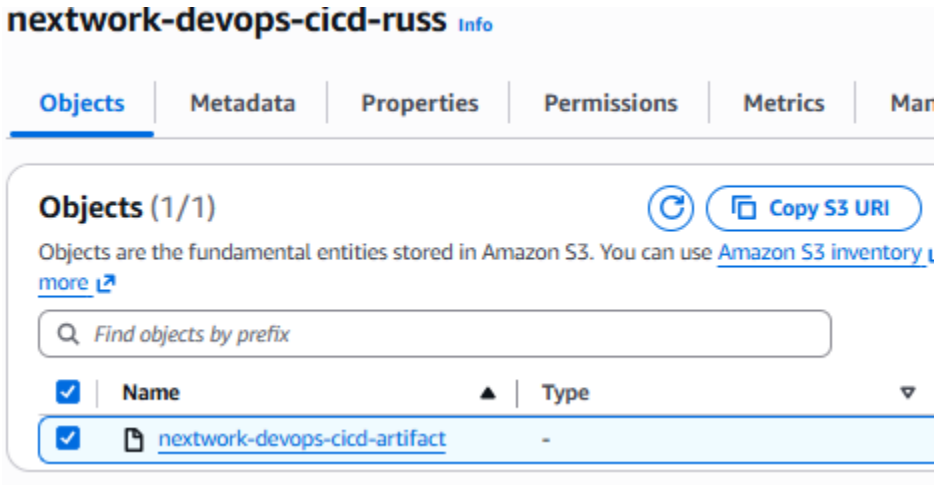
## Create and Verify Deployment

It's time to put everything together and deploy our web application to the EC2 instance!

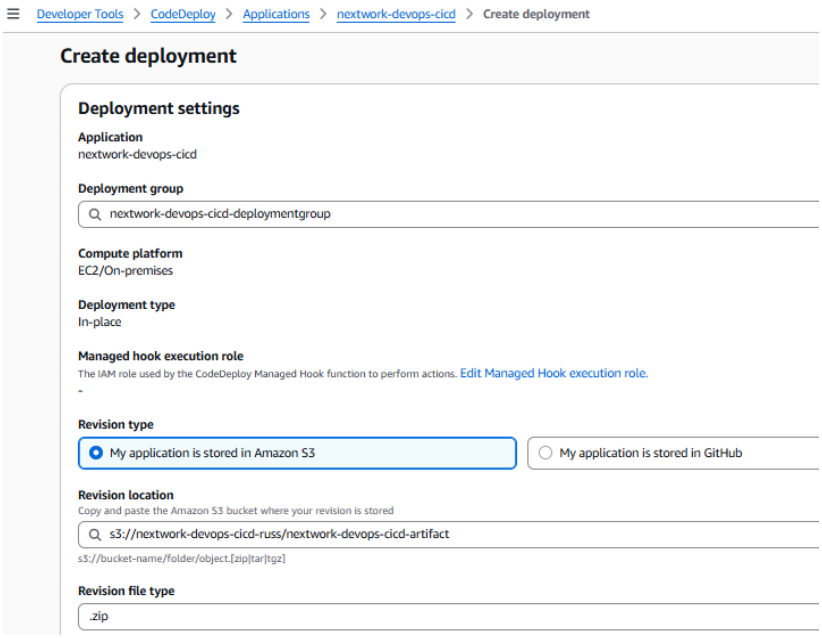


## Create Deployment

- In the deployment group details page, select **Create deployment**
- **Under Revision type,** make sure **My application is stored in Amazon S3** is selected. That's because our deployment artifact is inside an S3 bucket
- Head back to your S3 bucket called **nextwork-devops-cicd.**
- Click into the **nextwork-devops-cicd-artifact** build artifact
- Copy the file's S3 URI.

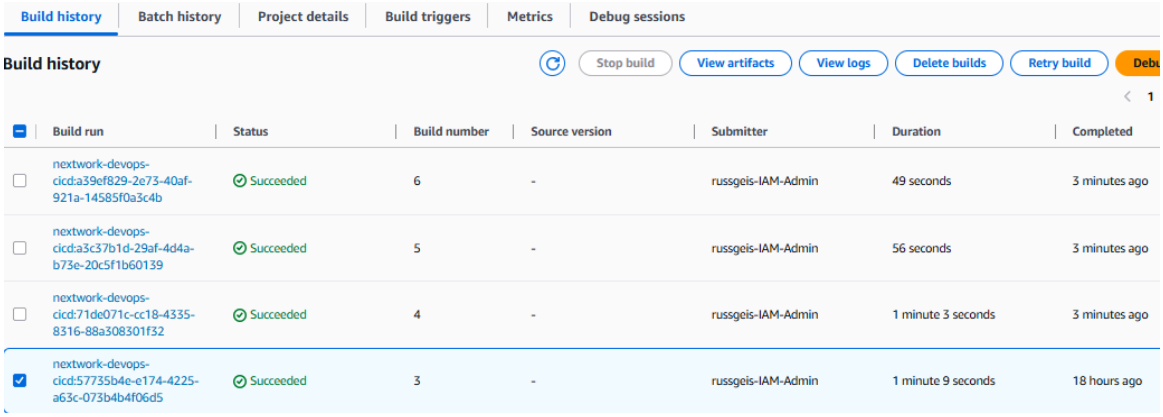



- Paste the S3 URI into the **Revision location** field in CodeDeploy**.**
- Select **.zip** as the Revision file type**.**




- Next, we'll leave **Additional deployment behavior settings** as default
- Select **Create deployment**.

- Select **Create deployment**.
- Scroll down to **Deployment lifecycle events** and monitor the events by clicking **View events**.
- See the lifecycle events progressing, such as BeforeInstall, ApplicationStart, etc. These are the events you defined in appspec.yml
- Whoops! After a few minutes of waiting, you might notice that you hit an error...
- Why do you think your deployment failed?
- Don't forget, CodeDeploy is deploying your web app by grabbing the latest **build artifact** from your S3 bucket.
- When was the last time you ran a build on CodeBuild? 🤔
- Aha - our last time running a build was **before** we added appspec and the deployment scripts!
- Because of this, your deployment instance isn't getting **any** of the scripts you've written - causing the error you're seeing now.
- Head back to your CodeBuild build project, and **rebuild** your project.




- Once your second build is a success, return to CodeDeploy and retry the deployment.

## Check Your Deployed App!

- Wait until the deployment status says **Success**
- Select the **Instance ID** in the **Deployment lifecycle events** panel. This takes you to the deployment EC2 instance you launched with CloudFormation

**Deployment lifecycle events**

| Instance ID | Duration | Status | Most recent event |
|---|---|---|---|
| i-04a58fb155e28abbd | 30 seconds | Succeeded | ValidateService |

- Get the **Public IPv4 DNS** of your EC2 instance from the EC2 console
- Open the **Public IPv4 DNS** in a web browser.
- It might take a minute or two for the application to become fully accessible after deployment
- Hint: Check whether your URL is using https - if it is, change it to http in the browser. This is because the EC2 instance's security group lets in connections on port 80

## SUCCESS

Not secure ec2-3-22-27-126.us-east-2.compute.amazonaws.com

# Hello Russell's World!

This is my project working!

If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o